The power of streaming data pipelines with Google Cloud products

In dynamic scenarios like the Galactic Grand Prix, streaming data pipelines are not just an advantage; they are essential for capturing immediate value and therefore must be built for resilience and a seamless user experience. This is accomplished by a design that ensures:

Click each button to expand the 4 critical streaming pipeline design elements.

Near instantaneous insights —

Streaming pipelines process information as it arrives, providing insights with sub-second latency. This means you get a real-time pulse on events, enabling immediate decision-making and rapid responses which are crucial for competitive environments where every second counts.

Freshness and relevance —

With streaming, your data is continuously updated ensuring that the information you're working with is always current. This leads to highly accurate, actionable information for live dashboards, anti-cheat detection, and optimizing the user experience without delay.

High-velocity —

These pipelines are inherently built to ingest and process data at unprecedented rates. They efficiently handle massive, continuous flows of information, preventing backlogs and ensuring that all valuable data is captured and processed as it occurs.

Event-driven responsiveness —

Many critical business moments are instantaneous—a financial transaction, an IoT sensor alert, or a customer's click. Streaming pipelines allow your systems to react immediately to these events, powering real-time applications and unlocking new opportunities for instant interactions and personalized experiences.

Building effective streaming data pipelines requires a clear understanding of why you're processing data in real-time.

Different business needs require different architectural patterns. Here, you'll explore the most common streaming use cases and some simple reference architectures before we dive into the Google Cloud products that bring them to life. Let's dive in!

Simple and complex ETL

Streaming Extract, Transform, Load (ETL) is a foundational use case for processing continuous data streams. It allows you to ingest, transform, and load data in real-time or near real-time, making it immediately available for analysis or further processing. Streaming ETL can be categorized into two main types based on the complexity of transformations.

This architecture shows how data moves from a source, through processing, to its final destination for analysis and storage.

Simple Streaming ETL

- **Use for:** Real-time analytics and dashboarding for events that are already in a structured or semi-structured format. This data primarily needs to be loaded into BigQuery for immediate querying, often with minimal transformation required.
- **How it happens:** Raw data is collected with Pub/Sub (or Managed Service for Apache Kafka), minimal transformations are applied with Dataflow, and then queried using SQL in BigQuery.
- **Examples:** Website clickstream data, IoT sensor readings, or application logs that are relatively clean and consistent in their format.
- **Galactic Grand Prix application:** Cymbal Gaming can track every click on the Galactic Grand Prix's tournament website in real-time to see which pages are most popular at any given moment.

Click to zoom the image.

This diagram depicts a multi-path data processing architecture that handles both streaming and batch data, and integrates with an external data source like Managed Service for Apache Kafka.

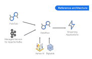
Complex Streaming ETL

- **Use for:** Scenarios where incoming data needs significant cleaning, joining with other datasets, aggregation, or schema restructuring *before* it's suitable for analytical queries. This requires more robust processing capabilities.
- **How it happens:** Transformations occur *before* the data is loaded into BigQuery.
- **Examples:** Calculating real-time Key Performance Indicators (KPIs), or preparing raw game event data for detailed reporting that requires a specific, transformed schema.
- **Galactic Grand Prix application:** Taking raw game events (like player movements or damage dealt) and transforming them into a unified structure (using Apache Beam), enriching them with player metadata (instead of knowing "player A moved", you know "a level 300 driver from Europe moved"), and aggregating them into real-time session statistics (like total distance driven per player per session, lap times, or number of collisions) before loading them for analysis.

Click to zoom the image.

Streaming AI/ML

The next use case you'll learn about is streaming AI/ML. This is about applying machine learning models directly to incoming data streams to generate immediate insights or actions. You can achieve this by performing common preprocessing methods (like embedding generation or creating n-grams) on incoming data, then applying an ML model for real-time inference.



This architecture illustrates the tools used in a machine learning data pipeline.

Key aspects of streaming AI/ML

- **Use for:** Applying ML models to data as it arrives to make instant decisions.
- **How it happens:** Perform preprocessing tasks like generating embeddings and bucketizing data, enrich events with a feature store, then call a model for inference.
- **Examples:** Real-time fraud detection, personalized recommendations, anomaly detection in operational data, and real-time risk assessment
- **Galactic Grand Prix application:** Cymbal Gaming can introduce a machine learning model to detect patterns indicative of cheating in real-time, flagging a player for review moments after a suspicious action.

Click to zoom the image.

Streaming applications

The use case of streaming applications leverages continuously updated data to build real-time, low-latency applications that require immediate access to information. These applications react instantly to streaming events, providing dynamic user experiences or backend services.

This simple diagram shows the tools used during streaming ingestion and data storage.

Key aspects of streaming applications

- **Use for:** Data with low latency requirements that must be processed and made available for applications with minimal delay.
- **How it happens:** Applications are powered by constantly refreshing data streams.
- **Examples:** Updating a user's real-time spending limit based on new transactions, tracking game scores in real-time for live leaderboards during an e-sports tournament, or powering a personalized news feed that updates as relevant articles are published.
- **Galactic Grand Prix application:** Cymbal Gaming can provide live, updated dashboards for tournament commentators showing player health, scores, and key game metrics as they unfold.

Click to zoom the image.

Reverse ETL

The final streaming use you'll learn about here is reverse ETL. Reverse ETL is a growing use case that involves activating data from your data warehouse or analytical system by sending processed insights back to operational systems or external applications. This closes the loop, putting analytical findings directly into the hands of operational teams or automated systems.

This diagram shows a reverse ETL architecture where data moves from a data warehouse back into operational business applications.

Key aspects of reverse ETL

- **Use for:** Leveraging valuable insights generated in BigQuery for operational uses.
- **How it happens:** Refined data is distributed to platforms like marketing automation tools, CRM (Customer Relationship Management) systems, or other operational databases.
- **Examples:** Sending customer segments defined in BigQuery to a marketing automation platform via Pub/Sub, updating user profiles in an operational database like Spanner or Bigtable based on analytical findings, or pushing transformed and aggregated data back into BigQuery for further analysis.
- **Galactic Grand Prix application:** After analyzing game data in BigQuery and identifying player segments (like "high-value viewers" or "players at risk of churn"), Cymbal Gaming engineers could use reverse ETL to push this segmentation data into a marketing platform to trigger targeted email campaigns or personalized in-game offers.

Click to zoom the image.

Let's review!

Match each use case on the left with the Google Cloud reference architecture that best implements it.

Real-time AI-powered data enrichment	Pub/Sub → BigQuery → Looker / Streaming Applications
Simplified real-time analytics dashboard	Pub/Sub → Dataflow → Feature Store → Vertex AI
Real-time AI feature engineering	BigQuery → Pub/Sub / Bigtable / Spanner → Streaming Applications
Serving layer for streaming applications	(Vertex AI / Bigtable / Feature Store) → Dataflow → Streaming Applications

SUBMIT

Now that you know more about the different use cases and reference architectures for streaming data on Google Cloud, the next step is to explore some of the key products that bring these architectures to life: Pub/Sub, Managed Service for Apache Kafka, Dataflow, BigQuery, and Bigtable.